

Machine Learning Lab

Introduction to Python for Machine Learning and Data Science

Department of Computer Engineering - Al-Azhar University - Egypt

OBJECTIVE

The objective of this lab is to provide a comprehensive introduction to Python programming, focusing on foundational concepts and data structures that serve as the base for machine learning and data science fields.

By the end of this lab, participants will have a solid understanding of Python variables, operations, lists, dictionaries, tuples, and sets. They will also be equipped with the knowledge of defining functions and utilizing other core Python coding techniques.

This lab serves as a crucial stepping stone for anyone aspiring to pursue machine learning and data science, as it establishes a strong foundation in Python, a primary language used in these domains.

MATERIALS

- Python 3.x installed on your computer.
- A code editor or integrated development environment (IDE) such as Jupyter Notebook.

PREREQUISITES

No prior knowledge of Python or NumPy is required. Basic programming concepts will be helpful.

INTRODUCTION TO PYTHON

Python is a high-level, general-purpose, and very popular programming language. Python programming language (latest Python 3) is being used in web development, Machine Learning applications, along with all cutting-edge technology in Software Industry.

Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber. . . etc.

The biggest strength of Python is huge collection of standard library which can be used for the following:

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like OpenCV, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia
- Scientific computing
- Text processing and many more

Python Operators

In Python programming, Operators in general are used to perform operations on values and variables. These are standard symbols used for the purpose of logical and arithmetic operations. We will look into different types of Python operators.

- OPERATORS: These are the special symbols. Eg- + , * , /, etc.
- OPERAND: It is the value on which the operator is applied.

There are various types of operators in Python:

1. Arithmetic Operators
2. Comparison Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operators
6. Identity Operators and Membership Operators

Arithmetic Operators in Python

Python Arithmetic operators are used to perform basic mathematical operations like addition, subtraction, multiplication, and division.

Precedence of Arithmetic Operators in Python The precedence of Arithmetic Operators in Python is as follows:

1. Parentheses
2. Exponentiation
3. Multiplication (Multiplication and division have the same precedence)

Operator	Description	Syntax
+	Addition: adds two operands	$x + y$
-	Subtraction: subtracts two operands	$x - y$
*	Multiplication: multiplies two operands	$x * y$
/	Division (float): divides the first operand by the second	x / y
//	Division (floor): divides the first operand by the second	$x // y$
%	Modulus: returns the remainder when the first operand is divided by the second	$x \% y$
**	Power: Returns first raised to power second	$x ** y$

4. Division

5. Addition (Addition and subtraction have the same precedence)

6. Subtraction

The modulus operator helps us extract the last digit/s of a number. For example:

- $x \% 10$ - > yields the last digit
- $x \% 100$ - > yield last two digits

Comparison Operators in Python

In Python Comparison of Relational operators compares the values. It either returns True or False according to the condition.

Operator	Description	Syntax
>	Greater than: True if the left operand is greater than the right	$x > y$
<	Less than: True if the left operand is less than the right	$x < y$
==	Equal to: True if both operands are equal	$x == y$
!=	Not equal to – True if operands are not equal	$x != y$
>=	Greater than or equal to True if the left operand is greater than or equal to the right	$x >= y$
<=	Less than or equal to True if the left operand is less than or equal to the right	$x <= y$

= is an assignment operator and == comparison operator.

Precedence of Comparison Operators in Python

In python, the comparison operators have lower precedence than the arithmetic operators. All the operators within comparison operators have same precedence order.

Logical Operators in Python

Python Logical operators perform Logical AND, Logical OR, and Logical NOT operations. It is used to combine conditional statements.

Operator	Description	Syntax
and	Logical AND: True if both the operands are true	x and y
or	Logical OR: True if either of the operands is true	x or y
not	Logical NOT: True if the operand is false	not x

Precedence of Logical Operators in Python The precedence of Logical Operators in Python is as follows:

1. Logical not
2. logical and
3. logical or

Bitwise Operators in Python

Python Bitwise operators act on bits and perform bit-by-bit operations. These are used to operate on binary numbers.

Operator	Description	Syntax
&	Bitwise AND	x & y
	Bitwise OR	x y
~	Bitwise NOT	~x
^	Bitwise XOR	x ^ y
>>	Bitwise right shift	x >>
<<	Bitwise left shift	x <<

Precedence of Bitwise Operators in Python The precedence of Bitwise Operators in python is as follows:

1. Bitwise NOT
2. Bitwise Shift
3. Bitwise AND
4. Bitwise XOR
5. Bitwise OR

Assignment Operators in Python

Python Assignment operators are used to assign values to the variables.

Operator	Description	Syntax
=	Assign the value of the right side of the expression to the left side operand	x = y + z
+=	Add AND: Add right-side operand with left-side operand and then assign to left operand	a+=b a=a+b
-=	Subtract AND: Subtract right operand from left operand and then assign to left operand	a-=b a=a-b
=	Multiply AND: Multiply right operand with left operand and then assign to left operand	a=b a=a*b
/=	Divide AND: Divide left operand with right operand and then assign to left operand	a/=b a=a/b
%=	Modulus AND: Takes modulus using left and right operands and assign the result to left operand	a%=b a=a%b
//=	Divide(floor) AND: Divide left operand with right operand and then assign the value(floor) to left operand	a//=b a=a//b
=	Exponent AND: Calculate exponent(raise power) value using operands and assign value to left operand	a=b a=a**b
&=	Performs Bitwise AND on operands and assign value to left operand	a&=b a=a&b
=	Performs Bitwise OR on operands and assign value to left operand	a =b a=a b
^=	Performs Bitwise xOR on operands and assign value to left operand	a^=b a=a^b
>>=	Performs Bitwise right shift on operands and assign value to left operand	a>>=b a=a>>b
<<=	Performs Bitwise left shift on operands and assign value to left operand	a<<=b a=a<<b

Identity Operators in Python

In Python, is and is not are the identity operators both are used to check if two values are located on the same part of the memory. Two variables that are equal do not imply that they are identical.

is True if the operands are identical
is not True if the operands are not identical

Membership Operators in Python

In Python, in and not in are the membership operators that are used to test whether a value or variable is in a sequence.

in True if value is found in the sequence
not in True if value is not found in the sequence

Ternary Operator in Python

In Python, Ternary operators also known as conditional expressions are operators that evaluate something based on a condition being true or false. It was added to Python in version 2.5.

It simply allows testing a condition in a single line replacing the multiline if-else making the code compact.

Syntax : [on true] if [expression] else [on false]

Lists, Dictionary, Tuples, and Sets

List in Python

The Python list is a sequence data type that is used to store the collection of data. Tuples and String are other types of sequence data types.

```
Var = ["Computer Programming", "Software Engineering", "Algorithms"]  
print(Var)
```

List comprehension

A Python list comprehension is made up of brackets carrying the expression, which is run for each element, as well as the for loop, which is used to iterate over the Python list's elements.

```
# Using list comprehension to iterate through loop  
List = [character for character in [1, 2, 3]]  
  
# Displaying list  
print(List)
```

Dictionary in Python

A dictionary in Python is a collection of key values, used to store data values like a map, which, unlike other data types holds only a single value as an element.

```
Dict = {1: 'Computer', 2: 'Electrical', 3: 'Civil'}  
print(Dict)
```

Python Dictionary Comprehension

Like List Comprehension, Python allows dictionary comprehension. We can create dictionaries using simple expressions. A dictionary comprehension takes the form key: value for (key, value) in iterable

```
# Lists to represent keys and values  
keys = ['a', 'b', 'c', 'd', 'e']  
values = [1, 2, 3, 4, 5]  
  
# but this line shows dict comprehension here  
myDict = { k:v for (k,v) in zip(keys, values)}  
  
# We can use below too  
# myDict = dict(zip(keys, values))  
print (myDict)
```

Tuples in Python

Tuple is a list-like collection of Python objects. A tuple stores a succession of values of any kind, which are indexed by integers.

```
var = ("Machine", "Learning")  
print(var)
```

Sets in Python

Python Set is an unordered collection of data types that can be iterated, mutated and contains no duplicate elements. The order of the elements in a set is unknown, yet it may contain several elements.

```
var = {"One", "Two", "One"}  
print(var)
```

In summary, lists, tuples, sets, and dictionaries are essential data structures in Python, each with its unique characteristics and use cases. Choosing the appropriate one depends on your specific needs, such as mutability, order, uniqueness, and the requirement for key-value associations. Understanding these distinctions allows you to select the most suitable data structure for your programming tasks.

Introduction to Python for Machine Learning and Data Science code

```
[1]: print("Python Coding")
```

Python Coding

```
[2]: print(type(12))
      print(type(12.5))
      print(type("Hello Python"))
```

<class 'int'>
<class 'float'>
<class 'str'>

```
[ ]:
```

```
[ ]:
```

```
[3]: # Program to check input from the user
```

```
num = int(input("Enter number :"))
print(num)
name1 = input("Enter name :")
print(name1)
```

```
# Printing type of input value
print("type of number", type(num))
print("type of name", type(name1))
```

Enter number :12

12

Enter name : Ahmed

Ahmed

type of number <class 'int'>

type of name <class 'str'>

```
[4]: ## Python program to show casting the variable
```

```
num = int(input("Enter a number: "))
print(num, " ", type(num))
```



```
floatNum = float(input("Enter a decimal number: "))
print(floatNum, "", type(floatNum))
```

Enter a number: 1

1 <class 'int'>

Enter a decimal number: 2

2.0 <class 'float'>

[5]: *### Typecasting the input to Float: To convert the input to float the following code will work out.*

```
# input
num1 = float(input())
num2 = float(input())

# printing the sum in float
print(num1 + num2)
```

56

26

82.0

[6]: *### Typecasting the input to String: All kinds of input can be converted to string type whether they are float or integer.*
We make use of keyword str for typecasting.
we can also take input string by just writing input() function by default it makes the input string

```
# input
string = str(input())

# output
print(string)

# Or by default
string_default = input()

# output
print(string_default)
```

Hello

Hello

Engineering

Engineering

```
[7]: # Python program showing how to
# multiple input using split

# taking two inputs at a time
x, y = input("Enter two values: ").split()
print("Number of boys: ", x)
print("Number of girls: ", y)

# taking three inputs at a time
x, y, z = input("Enter three values: ").split()
print("Total number of students: ", x)
print("Number of boys is : ", y)
print("Number of girls is : ", z)

# taking two inputs at a time
a, b = input("Enter two values: ").split()
print("First number is {} and second number is {}".format(a, b))

# taking multiple inputs at a time
# and type casting using list() function
x = list(map(int, input("Enter multiple values: ").split()))
print("List of students: ", x)
```

Enter two values: Computer Engineerirng
 Number of boys: Computer
 Number of girls: Engineeirng
 Enter three values: Systems Computer Engineering
 Total number of students: Systems
 Number of boys is : Computer
 Number of girls is : Engineering
 Enter two values: One Two
 First number is One and second number is Two
 Enter multiple values: 1 2 3 4 5 6 7
 List of students: [1, 2, 3, 4, 5, 6, 7]

Using List comprehension :

List comprehension is an elegant way to define and create a list in Python. We can create lists just like mathematical statements in one line only. It is also used in getting multiple inputs from a user.

```
[12]: # Python program showing how to take multiple input using List comprehension

# taking two input at a time
x, y = [int(x) for x in input("Enter two values: ").split()]
print("First Number is: ", x)
print("Second Number is: ", y)

# taking three input at a time
x, y, z = [int(x) for x in input("Enter three values: ").split()]
```

```

print("First Number is: ", x)
print("Second Number is: ", y)
print("Third Number is: ", z)

# taking two inputs at a time
x, y = [int(x) for x in input("Enter two values: ").split()]
print("First number is {} and second number is {}".format(x, y))

# taking multiple inputs at a time
x = [int(x) for x in input("Enter multiple values: ").split()]
print("Number of list is: ", x)

```

Enter two values: 12 2
 First Number is: 12
 Second Number is: 2
 Enter three values: 12 5 6
 First Number is: 12
 Second Number is: 5
 Third Number is: 6
 Enter two values: 23 84
 First number is 23 and second number is 84
 Enter multiple values: 15 48 65 14
 Number of list is: [15, 48, 65, 14]

Note: The above examples take input separated by spaces. In case we wish to take input separated by comma (,), we can use the following:

```

[15]: # taking multiple inputs at a time separated by comma
x = [int(x) for x in input("Enter multiple value: ").split(",")]
print("Number of list is: ", x)

```

Enter multiple value: 1,2,58,78
 Number of list is: [1, 2, 58, 78]

Print without newline in Python 3.x In Python 3.x, the print() function behaves slightly differently from Python 2.x. To print without a newline in Python 3. x, you can use the end parameter of the print() function.

```

[16]: # Python 3 code for printing
# on the same line printing
# geeks and geeksforgeeks
# in the same line

print("geeks", end = " ")
print("geeksforgeeks")

# array
a = [1, 2, 3, 4]

```

```
# printing a element in same
# line
for i in range(4):
    print(a[i], end = " ")
```

geeks geeksforgeeks
1 2 3 4

Print without newline in Python 3.x without using For Loop In Python 3. x, you can print without a newline without using a for loop by using the sep parameter of the print() function. The sep parameter specifies the separator to be used between multiple items when they are printed.

[17]: *# Print without newline in Python 3.x without using for loop*

```
l = [1, 2, 3, 4, 5, 6]

# using * symbol prints the list
# elements in a single line
print(*l)
```

1 2 3 4 5 6

Print without newline Using Python sys module

[18]: `import sys`

```
sys.stdout.write("Website A ")
sys.stdout.write("is best website for coding")
```

Website A is best website for coding

[18]: 26

Python end parameter in print()

[19]: `print('G','F', sep='', end='')`
`print('G')`
#\n provides new line after printing the year
`print('09','12','2016', sep='-', end='\n')`

```
print('Red','Green','Blue', sep=',', end='@')
print("geeksforgeeks")
```

GFG

09-12-2016

Red,Green,Blue@geeksforgeeks

[20]: `name = "Alice"`
`age = 30`
`print("My name is", name, "and I am", age, "years old.", end=" ")`
`print("Nice to meet you!")`

My name is Alice and I am 30 years old. Nice to meet you!

```
[21]: # using a comma separator
print('apples', 'oranges', 'bananas', sep=', ')
# output: apples, oranges,bananas

# using a semicolon separator
print('one', 'two', 'three', sep=';')
# output: one;two;three

# using an emoji separator
print('????', '????', '????', sep='???')
# output: ????????????????????
```

apples, oranges, bananas
one;two;three
????????????????????

```
[22]: # using a comma separator
print('apples', 'oranges', 'bananas', sep=', ')
# output: apples, oranges,bananas

# using a semicolon separator
print('one', 'two', 'three', sep=';')
# output: one;two;three

# using an emoji separator
print('????', '????', '????', sep='???')
# output: ????????????????????
```

apples, oranges, bananas
one;two;three
????????????????????

Formatting Output using The Format Method

```
[23]: print("This is the {} Engineering {}".format("Computer", "Department"))

# using format() method and referring a position of the object
print('{0} and {1}'.format("Computer", "Engineering"))

print('{1} and {0}'.format("Hello", "Python"))

x = 2
y = 3

print(f'{x} and {y}')
```

This is the Computer Engineering Department
Computer and Engineering

Python and Hello

2 and 3

Python Operators

– Types of Operators in Python

Arithmetic Operators

Comparison Operators

Logical Operators

Bitwise Operators

Assignment Operators

Identity Operators and Membership Operators

```
[24]: # Examples of ArithmeticOperator
a = 9
b = 4

# Addition of numbers
add = a + b

# Subtraction of numbers
sub = a - b

# Multiplication of number
mul = a * b

# Modulo of both number
mod = a % b

# Power
p = a ** b

# print results
print(add)
print(sub)
print(mul)
print(mod)
print(p)
```

13

5

36

1

6561

[25]: *# Examples of Relational Operators*

```
a = 13
b = 33

# a > b is False
print(a > b)

# a < b is True
print(a < b)

# a == b is False
print(a == b)

# a != b is True
print(a != b)

# a >= b is False
print(a >= b)

# a <= b is True
print(a <= b)
```

False

True

False

True

False

True

[26]: *# Examples of Logical Operator*

```
a = True
b = False

# Print a and b is False
print(a and b)

# Print a or b is True
print(a or b)

# Print not a is False
print(not a)
```

False

True

False

[27]: *# Examples of Bitwise operators*

```
a = 10
```

```

b = 4

# Print bitwise AND operation
print(a & b)

# Print bitwise OR operation
print(a | b)

# Print bitwise NOT operation
print(~a)

# print bitwise XOR operation
print(a ^ b)

# print bitwise right shift operation
print(a >> 2)

# print bitwise left shift operation
print(a << 2)

```

0
14
-11
14
2
40

[28]: *# Examples of Assignment Operators*

```

a = 10

# Assign value
b = a
print(b)

# Add and assign value
b += a
print(b)

# Subtract and assign value
b -= a
print(b)

# multiply and assign
b *= a
print(b)

# bitwise lishift operator

```



```
b <= a
print(b)
```

10
20
10
100
102400

[29]: *#Example Identity Operators in Python*

```
a = 10
b = 20
c = a

print(a is not b)
print(a is c)
```

True
True

[30]: *# Python program to illustrate
not 'in' operator*

```
x = 24
y = 20
list = [10, 20, 30, 40, 50]

if (x not in list):
    print("x is NOT present in given list")
else:
    print("x is present in given list")

if (y in list):
    print("y is present in given list")
else:
    print("y is NOT present in given list")
```

x is NOT present in given list
y is present in given list

[31]: *# Program to demonstrate conditional operator*
a, b = 10, 20

```
# Copy value of a in min if a < b else copy b
min = a if a < b else b

print(min)
```

10

```
[32]: # Examples of Operator Precedence

# Precedence of '+' & '*'
expr = 10 + 20 * 30
print(expr)

# Precedence of 'or' & 'and'
name = "Alex"
age = 0

if name == "Alex" or name == "John" and age >= 2:
    print("Hello! Welcome.")
else:
    print("Good Bye!!")
```

610

Hello! Welcome.

```
[33]: # Examples of Operator Associativity

# Left-right associativity
# 100 / 10 * 10 is calculated as
# (100 / 10) * 10 and not
# as 100 / (10 * 10)
print(100 / 10 * 10)

# Left-right associativity
# 5 - 2 + 3 is calculated as
# (5 - 2) + 3 and not
# as 5 - (2 + 3)
print(5 - 2 + 3)

# left-right associativity
print(5 - (2 + 3))

# right-left associativity
# 2 ** 3 ** 2 is calculated as
# 2 ** (3 ** 2) and not
# as (2 ** 3) ** 2
print(2 ** 3 ** 2)
```

100.0

6

0

512

```
[34]: # Creating a String
String1 = "Hello Engineering"
```

```

print("Initial String: ")
print(String1)

# Printing 3rd character
print("\nSlicing characters from 3-12: ")
print(String1[3])

# Printing characters between
# 3rd and 2nd last character
print("\nSlicing characters between " +
      "3rd and 2nd last character: ")
print(String1[3:-2])

```

Initial String:

Hello Engineering

Slicing characters from 3-12:

l

Slicing characters between 3rd and 2nd last character:

lo Engineei

Create a simple calculator that can perform basic arithmetic operations like addition, subtraction, multiplication, or division depending upon the user input. Approach :

The user chooses the desired operation. Options 1, 2, 3, and 4 are valid. Two numbers are taken and an if...elif...else branching is used to execute a particular section. Using functions add(), subtract(), multiply() and divide() evaluate respective operations.

[35]: *# Python program for simple calculator*

```

# Function to add two numbers
def add(num1, num2):
    return num1 + num2

# Function to subtract two numbers
def subtract(num1, num2):
    return num1 - num2

# Function to multiply two numbers
def multiply(num1, num2):
    return num1 * num2

# Function to divide two numbers
def divide(num1, num2):
    return num1 / num2

print("Please select operation -\n" \

```

```

    "1. Add\n" \
    "2. Subtract\n" \
    "3. Multiply\n" \
    "4. Divide\n")

# Take input from the user
select = int(input("Select operations form 1, 2, 3, 4 :"))

number_1 = int(input("Enter first number: "))
number_2 = int(input("Enter second number: "))

if select == 1:
    print(number_1, "+", number_2, "=",
          add(number_1, number_2))

elif select == 2:
    print(number_1, "-", number_2, "=",
          subtract(number_1, number_2))

elif select == 3:
    print(number_1, "*", number_2, "=",
          multiply(number_1, number_2))

elif select == 4:
    print(number_1, "/", number_2, "=",
          divide(number_1, number_2))
else:
    print("Invalid input")

```

Please select operation –

1. Add
2. Subtract
3. Multiply
4. Divide

Select operations form 1, 2, 3, 4 :2

Enter first number: 25

Enter second number: 2

25 - 2 = 23

We define a function is even that checks whether a number is even or odd. We have a list of numbers from 1 to 10. We use a list comprehension with a lambda function to filter even numbers into the even numbers list. We use a list comprehension with a lambda function to square each number and store the results in the squared numbers list. We define a factorial function and use list comprehension to calculate the factorial of each number and store the results in the factorials list. This code demonstrates the use of control statements, functions, list comprehensions, and lambda functions in one comprehensive example.

```
[36]: # Function to check if a number is even or odd
def is_even(num):
    return num % 2 == 0

# List of numbers
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Use list comprehension and a lambda function to filter even numbers
even_numbers = [num for num in numbers if is_even(num)]

# Use list comprehension and a lambda function to square each number
squared_numbers = [(lambda x: x**2)(num) for num in numbers]

# Use list comprehension to get the factorial of each number
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

factorials = [factorial(num) for num in numbers]

# Print the results
print("Original Numbers:", numbers)
print("Even Numbers:", even_numbers)
print("Squared Numbers:", squared_numbers)
print("Factorials:", factorials)
```

Original Numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Even Numbers: [2, 4, 6, 8, 10]

Squared Numbers: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

Factorials: [1, 2, 6, 24, 120, 720, 5040, 40320, 362880, 3628800]

As for using the zip function, it's often used to combine two or more iterables element-wise. Here's an example:

```
[41]: names = ["Alice", "Bob", "Charlie"]
scores = [95, 87, 92]

# Using zip to combine names and scores element-wise
name_score_pairs = zip(names, scores)

# Printing the result
for name, score in name_score_pairs:
    print(f"{name}: {score}")
```

Alice: 95

Bob: 87

Charlie: 92

Functions in Python

```
[42]: # A simple Python function
def welcome():
    print("Welcome to Computer Enigneering")

# Driver code to call a function
welcome()
```

Welcome to Computer Enigneering

```
[43]: # A simple Python function to check
# whether x is even or odd
def evenOdd(x):
    if (x % 2 == 0):
        print("even")
    else:
        print("odd")

# Driver code to call the function
evenOdd(2)
evenOdd(3)
```

even
odd

```
[44]: # Python program to
# demonstrate return statement
def add(a, b):

    # returning sum of a and b
    return a + b

def is_true(a):

    # returning boolean of a
    return bool(a)

# calling function
res = add(2, 3)
print("Result of add function is {}".format(res))

res = is_true(2<5)
print("\nResult of is_true function is {}".format(res))
```

Result of add function is 5

Result of is_true function is True

```
[45]: # print first 5 integers
# using python range() function
for i in range(5):
    print(i, end=" ")
print()
```

0 1 2 3 4

Python Filter Function

The filter() method filters the given sequence using a function that examines each element in the sequence to see if it is true or false.

```
[46]: # function that filters vowels
def fun(variable):
    letters = ['a', 'e', 'i', 'o', 'u']
    if (variable in letters):
        return True
    else:
        return False

# sequence
sequence = ['g', 'e', 'e', 'j', 'k', 's', 'p', 'r']

# using filter function
filtered = filter(fun, sequence)

print("The filtered letters are:")
for s in filtered:
    print(s)
```

The filtered letters are:

e
e

Python Reduce Function

The reduce function is used to apply a certain function to all of the list components indicated in the sequence sent along.

```
[47]: from functools import reduce

nums = [1, 2, 3, 4]
ans = reduce(lambda x, y: x + y, nums)
print(ans)
```

10

Python Lambda

Python Lambda Functions are anonymous, which means they have no name. As we already know,

the def keyword is used to define a normal function in Python. The lambda keyword in Python is used to declare an anonymous function.

```
[48]: calc = lambda num: "Even number" if num % 2 == 0 else "Odd number"
      print(calc(20))
```

Even number

*args and **kwargs in Python

The *args and *kwargs keywords allow functions to take variable-length parameters. The number of non-keyworded arguments and the action that can be performed on the tuple are specified by the *args. *kwargs, on the other hand, pass a variable number of keyword arguments dictionary to function, which can then do dictionary operations.

```
[49]: def Function1(arg1, arg2, arg3):
      print("arg1:", arg1)
      print("arg2:", arg2)
      print("arg3:", arg3)

      # Now we can use *args or **kwargs to
      # pass arguments to this function :
      args = ("Funct1", "Funct2", "Funct3")
      Function1(*args)

      kwargs = {"arg1": "Funct1", "arg2": "Funct2", "arg3": "Funct3"}
      Function1(**kwargs)
```

arg1: Funct1
arg2: Funct2
arg3: Funct3
arg1: Funct1
arg2: Funct2
arg3: Funct3

Try and Except Statement

In Python, Try and except statements are used to catch and manage exceptions. Statements that can raise exceptions are kept inside the try clause and the statements that handle the exception are written inside except clause.

```
[50]: a = [1, 2, 3]
      try:
          print ("Second element = %d" %(a[1]))

          # Throws error since there are only 3 elements in array
          print ("Fourth element = %d" %(a[3]))
```



```
except:
    print ("An error occurred")
```

Second element = 2

An error occurred

File Handling in Python

Python too supports file handling and allows users to handle files i.e., to read and write files, along with many other file handling options, to operate on files.

```
[51]: import os

def create_file(filename):
    try:
        with open(filename, 'w') as f:
            f.write("Hello, world!\n")
        print("File " + filename + " created successfully.")
    except IOError:
        print("Error: could not create file " + filename)

def read_file(filename):
    try:
        with open(filename, 'r') as f:
            contents = f.read()
            print(contents)
    except IOError:
        print("Error: could not read file " + filename)

def append_file(filename, text):
    try:
        with open(filename, 'a') as f:
            f.write(text)
        print("Text appended to file " + filename + " successfully.")
    except IOError:
        print("Error: could not append to file " + filename)

def rename_file(filename, new_filename):
    try:
        os.rename(filename, new_filename)
        print("File " + filename + " renamed to " +
              new_filename + " successfully.")
    except IOError:
        print("Error: could not rename file " + filename)

def delete_file(filename):
    try:
        os.remove(filename)
```

```

        print("File " + filename + " deleted successfully.")
    except IOError:
        print("Error: could not delete file " + filename)

if __name__ == '__main__':
    filename = "example.txt"
    new_filename = "new_example.txt"

    create_file(filename)
    read_file(filename)
    append_file(filename, "This is some additional text.\n")
    read_file(filename)
    rename_file(filename, new_filename)
    read_file(new_filename)
    delete_file(new_filename)

```

File example.txt created successfully.
Hello, world!

Text appended to file example.txt successfully.
Hello, world!
This is some additional text.

File example.txt renamed to new_example.txt successfully.
Hello, world!
This is some additional text.

File new_example.txt deleted successfully.
Example on List

```

[52]: # Creating a list
fruits = ["apple", "banana", "cherry", "date"]

# Accessing elements
print("First fruit:", fruits[0])
print("Last fruit:", fruits[-1])

# Slicing
print("Sliced fruits:", fruits[1:3])

# Modifying elements
fruits[1] = "grape"
print("Modified list:", fruits)

# Adding elements
fruits.append("fig")

```

```

print("After adding a fruit:", fruits)

# Removing elements
removed_fruit = fruits.pop(2)
print("Removed fruit:", removed_fruit)
print("Updated list after removal:", fruits)

```

First fruit: apple
 Last fruit: date
 Sliced fruits: ['banana', 'cherry']
 Modified list: ['apple', 'grape', 'cherry', 'date']
 After adding a fruit: ['apple', 'grape', 'cherry', 'date', 'fig']
 Removed fruit: cherry
 Updated list after removal: ['apple', 'grape', 'date', 'fig']

Example on Tuples

```

[53]: # Creating a tuple
colors = ("red", "green", "blue")

# Accessing elements
print("First color:", colors[0])

# Tuple unpacking
a, b, c = colors
print("Unpacked values:", a, b, c)

```

First color: red
 Unpacked values: red green blue

Example on Lists

```

[54]: # Creating a set
fruits_set = {"apple", "banana", "cherry"}

# Adding and removing elements
fruits_set.add("date")
fruits_set.remove("banana")

# Checking membership
print("Is 'cherry' in the set?", "cherry" in fruits_set)

# Set operations
vegetables_set = {"carrot", "spinach", "banana"}
union_set = fruits_set.union(vegetables_set)
intersection_set = fruits_set.intersection(vegetables_set)

print("Union of sets:", union_set)
print("Intersection of sets:", intersection_set)

```

Is 'cherry' in the set? True

Union of sets: {'cherry', 'spinach', 'banana', 'date', 'apple', 'carrot'}

Intersection of sets: set()

Example on Dictionary

```
[55]: # Creating a dictionary
student = {
    "name": "Alice",
    "age": 20,
    "major": "Computer Science"
}

# Accessing values
print("Name:", student["name"])
print("Age:", student["age"])

# Modifying values
student["age"] = 21
student["year"] = "Senior"

# Iterating through keys and values
for key, value in student.items():
    print(key, ":", value)

# Removing a key-value pair
del student["major"]
```

Name: Alice

Age: 20

name : Alice

age : 21

major : Computer Science

year : Senior

```
[56]: # A generator function that yields 1 for first time,
# 2 second time and 3 third time
def simpleGeneratorFun():
    yield 1
    yield 2
    yield 3

# Driver code to check above generator function
for value in simpleGeneratorFun():
    print(value)
```

1

2

3

```
[57]: # A Python program to demonstrate use of
# generator object with next()

# A generator function
def simpleGeneratorFun():
    yield 1
    yield 2
    yield 3

# x is a generator object
x = simpleGeneratorFun()

# Iterating over the generator object using next

# In Python 3, _next_()
print(next(x))
print(next(x))
print(next(x))
```

```
1
2
3
```

```
[58]: # A simple generator for Fibonacci Numbers
def fib(limit):

    # Initialize first two Fibonacci Numbers
    a, b = 0, 1

    # One by one yield next Fibonacci Number
    while a < limit:
        yield a
        a, b = b, a + b

# Create a generator object
x = fib(5)

# Iterating over the generator object using next
# In Python 3, _next_()
print(next(x))
print(next(x))
print(next(x))
print(next(x))
print(next(x))

# Iterating over the generator object using for
# in loop.
print("\nUsing for in loop")
```

```
for i in fib(5):  
    print(i)
```

0
1
1
2
3

Using for in loop

0
1
1
2
3

```
[59]: # generator expression  
generator_exp = (i * 5 for i in range(5) if i % 2 == 0)  
  
for i in generator_exp:  
    print(i)
```

0
10
20

```
[60]: calc = lambda num: "Even number" if num % 2 == 0 else "Odd number"  
  
print(calc(20))
```

Even number

```
[ ]:
```

EXERCISES - ToDo IN THE LAB

1. Write a program to print all the numbers between 1000 and 2000 which are divisible by 7 but are not a multiple of 5.
2. Write a program to calculate factorial of a number.
3. Write a program that takes a list of numbers and returns a new list containing only the even numbers.(Use List Comprehension)

ASSIGNMENT

1. Write a Python function that finds the minimum common element between two lists
2. Write a Python function that calculates the area and circumference of a circle given its radius.(Hint: you can import math library)
3. Write a Python function that sorts a list of dictionaries by a specific key.